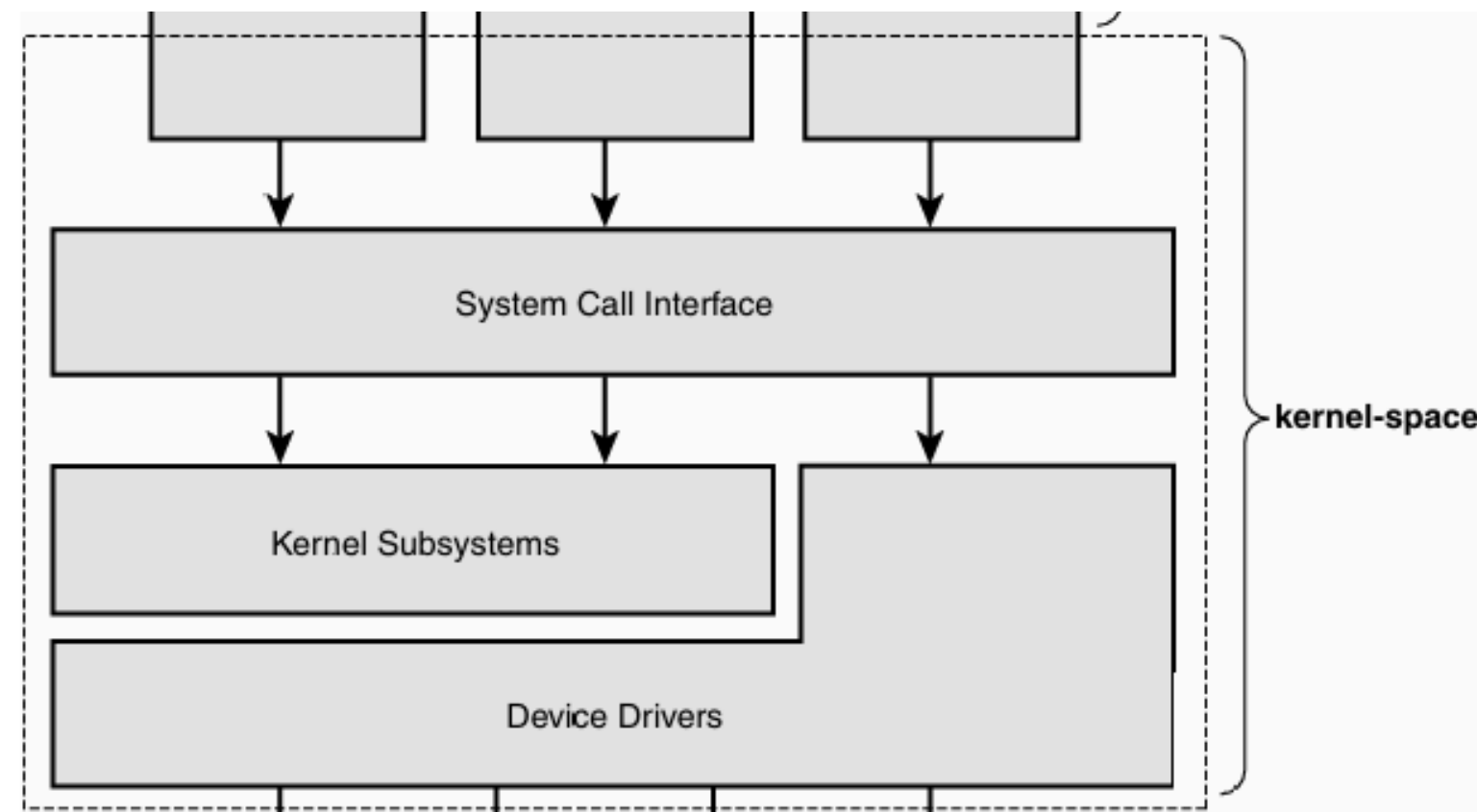


Linux Kernel Development:

Kernel Module

Within Kernel Space



Although Linux kernel uses **monolith** software architecture (in a sense that it share the same address space). Linux kernel manages resources, such as CPU time and memory, through multiple subsystems that work together, each responsible for manage different resource. The example subsystems are:

- Process Scheduler
- Memory Management
- Power management
- Virtual File System
- Locking

and more. You can find out more from: <https://docs.kernel.org/subsystem-apis.html>

Kernel Building On ubuntu

Install build requirements

```
$ sudo apt build-dep -y linux linux-image-unsigned-$(uname -r) && sudo apt install -y fakeroot llvm  
libncurses-dev dwarves
```

Download the sources

```
$ git clone https://github.com/torvalds/linux.git or $ apt source linux-image-unsigned-$(uname -r)
```

Config the kernel or modify the source file

```
$ make menuconfig or debian/rules editconfigs
```

Kernel Building On ubuntu

Build Kernel

```
$ make -j$(nproc)
```

Packaging

```
$ make -j$(nproc) bindeb-pkg
```

```
$ debian/rules binary
```

Install

```
$ sudo dpkg -i linux-headers-<kernel version>*_all.deb \  
sudo dpkg -i linux-headers-<kernel version>-<generic or derivative>*.deb \  
sudo dpkg -i linux-image-unsigned-<kernel version>-<generic or derivative>*.deb \  
sudo dpkg -i linux-modules-<kernel version>-<generic or derivative>*.deb
```

- However, building kernel this way can take anywhere from **15 minutes (high-end workstation) to several hours** which is **not ideal for development** where kernel need to be recompile repeatedly.
- The solution is fairly simple. After the first build any build afterward will only compile the changed part or code. This system often refered as incremental build.

Kernel Module

- Kernel Module are code segment capable of dynamic **loading and unloading** via the use **insmod, rmmod**
- This allow the development to be in some part of code to be significantly faster as it does not require developer to recompile the code and reboot every time the code is modified
- Note that the kernel module only support some part of the code such as device driver, network extension, and etc. Most of the core system still require you to rebuild the entire kernel.

Hello World In kernel space

Prerequisite Install

```
$ sudo apt-get install build-essential kmod
```

list the kernel module currently running

```
$ lsmod
```

hello-1.c

```
#include <linux/module.h> /* Needed by all modules */
#include <linux/printk.h> /* Needed for pr_info() */
int init_module(void)
{
    pr_info("Hello world 1.\n");
    return 0;
}
void cleanup_module(void)
{
    pr_info("Goodbye world 1.\n");
}
MODULE_LICENSE("GPL");
```

Build Kernel module

Makefile

```
obj-m += hello-1.o
PWD := <current path>
all:
    $(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
    $(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Build

```
make
```

Load module

```
sudo insmod hello.ko
```

Remove Module

```
sudo rmmod hello.ko
```

Check kernel log

```
sudo dmesg | tail -10
```

Demo